Competições de Algoritmos e Programação UFRJ

Universidade Federal do Rio de Janeiro

O céu ganhou um novo anjinho. Que Chris cuide de Chen com muito carinho S2

Enzo Vieira, Caio David, Luís Rafael

Agosto, 2024

Sumário

Capítulo 1

C++ e Biblioteca STD

1.1 Template

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
typedef long long ll;
const int mxn = 1e5+5;

int cmp_double(double a, double b=0, double eps=1e-9){
    return a + eps > b ? b + eps > a ? 0 : 1 : -1;
}

int main(){
    ios_base::sync_with_stdio(false), cin.tie(nullptr);

    return 0;
}
```

1.2 Ordered Set

```
#include <ext/pb_ds/tree_policy.hpp>
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

typedef tree<int, null_type, less<int>, rb_tree_tag,
tree_order_statistics_node_update> ordered_set;
//less_equal<int> and upper_bound(not find) for multi_orderedset
```

Membros de ordered_set:
find_by_order(p) //Retorna um ponteiro para o p-ésimo elemento do set. Se p é maior que o tamanho de n, retorna o fim do set. Index em 0
order_of_key(v) //Retorna quantos elementos são menores que v.

Mesmo set com operações de find_by_order e order_of_key.

1.3 Bitset

bitset<mxn> bs;
Complexidade: O(mxn/32)

```
#pragma GCC target("popcnt") //faster popcount
```

Membros: **empty()**, **count()**, **to_string()**, **to_ulong()**, **to_ullong()**.
set(i) → Seta o bit i para 1 / Seta todos os bits para 1, se set()
reset(i) → Seta o bit i para 0 / Seta todos os bits para 0, se reset()
flip(i) → Alterna o bit i / Alterna todos os bits, se flip()
operador »
operador «
operador &
operador |
operador ^
operador ~
operador ==
operador !=
__builtin_popcount(x) //Conta o número de bits ligados, ll no final para long long

1.4 Unordered_Map

```
//use map.reserve(1e7) to go faster

struct custom_hash{
    size_t operator()(uint64_t x) const{
        static const uint64_t FIXED_RANDOM =
        chrono::steady_clock::now().time_since_epoch().count();
        return x+FIXED_RANDOM;
    }
};

struct custom_hash{
    size_t operator()(uint64_t x) const{
        static const uint64_t FIXED_RANDOM =
        chrono::steady_clock::now().time_since_epoch().count();
        x^=FIXED_RANDOM;
        return x^(x >> 16);
    }
};

struct custom_hash{
    static uint64_t splitmix64(uint64_t x){
        x+=0x9e3779b97f4a7c15;
        x = (x^(x >> 30))*0xbf58476d1ce4e5b9;
        x = (x^(x >> 27))*0x94d049bb133111eb;
        return x^(x >> 31);
    }

    size_t operator()(uint64_t x) const{
        static const uint64_t FIXED_RANDOM =
        chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x+FIXED_RANDOM);
    }
};
unordered_map<int, int, custom_hash> mapa;
```

Capítulo 2

Estruturas de Dados

2.1 Sparse Table

```
struct SparseTable{
  int K = 25, n;
  vector <vector<int>> st; //st[i][j] = min on range [j, j + 2^i-1]
  vector <int> lg2; //lg2[i] = floor(log2(i))

  SparseTable(vector <int> arr){
    n = arr.size();
    st.resize(K+1);
    for(auto& v : st) v.resize(n);

    st[0] = arr;
    for(int i = 1; i <= K; i++){
      for(int j = 0; j + (1 << i) - 1 < n; j++){
        st[i][j] = min(st[i-1][j], st[i-1][j + (1 << (i - 1))]);
      }
    }

    lg2.resize(n+1);
    lg2[1] = 0;
    for(int i = 2; i <= n; i++){
      lg2[i] = lg2[i/2] + 1;
    }
  }

  int query(int l, int r){
    int i = lg2[r-l+1];
    return min(st[i][l], st[i][r-(1<<i)+1]);
  }
};
```

2.2 PrefixSum2D

```
struct PrefixSum2D{
  vector <vector<int>> pref;

  PrefixSum2D(int n, int m, vector<vector<int>> mat){
    pref.resize(n+1);
    for(auto& v : pref) v.resize(m+1, 0);

    for(int i = 1; i <= n; i++){
      for(int j = 1; j <= m; j++){
        pref[i][j] = pref[i-1][j] + pref[i][j-1]
        - pref[i-1][j-1] + mat[i-1][j-1];
      }
    }
  }

  int get_sum(int rowl, int rowr, int coll, int colr){ //0 indexado
    rowl += 1, rowr += 1, coll += 1, colr += 1;
    if(rowl > rowr) swap(rowl, rowr);
    if(coll > colr) swap(colr, coll);
    return pref[rowr][colr] - pref[rowl-1][colr]
    - pref[rowr][coll-1] + pref[rowl-1][coll-1];
  }
};
```

2.3 SegTree

```
struct Node{
  int val;
  Node operator+(Node other) const{
    return {this->val + other.val};
  }
  Node operator=(int x){
    return {this->val = x};
  }
};

struct SegTree{ //segtree 0 indexada sem lazy
  int n;
  Node neutral = {0};
  vector <Node> t;

  SegTree(vector <int> a){
    n = 1;
    while(n < a.size()){
      n *= 2;
    }
    t.assign(2*n, neutral);
    build(a, 1, 0, n-1);
  }

  void build(vector <int>& a, int v, int tl, int tr) {
    if(tl >= a.size()){
        t[v] = neutral;
    }else if (tl == tr) {
      t[v] = a[tl];
    } else {
      int tm = (tl + tr) / 2;
      build(a, v*2, tl, tm);
      build(a, v*2+1, tm+1, tr);
      t[v] = t[v*2] + t[v*2+1];
    }
  }

  Node query(int l, int r){
    return query(1, 0, n-1, l, r);
  }

  Node query(int v, int tl, int tr, int l, int r){
    if(l > tr || r < tl){
      return neutral;
    }
    if(l <= tl && tr <= r){
      return t[v];
    }
    int tm = (tl + tr) / 2;
    return query(v*2, tl, tm, l, r)
      + query(v*2+1, tm+1, tr, l, r);
  }

  void update(int pos, int val){
    update(1, 0, n-1, pos, val);
  }

  void update(int v, int tl, int tr, int pos, int new_val){
    if (tl == tr) {
      t[v] = new_val;
    } else {
      int tm = (tl + tr) / 2;
      if (pos <= tm)
        update(v*2, tl, tm, pos, new_val);
      else
        update(v*2+1, tm+1, tr, pos, new_val);
      t[v] = t[v*2] + t[v*2+1];
    }
  }
};
```

2.4 DSU

```
template<typename T> struct DSU
{
    int n;
    vector<int> pai, rank;

    DSU(int n) : n(n), pai(n+1), rank(n+1,1){
        for(int i = 1; i <=n; i++){
            pai[i] = i;
```

```
        }
    }

    int find(int a){
        if(pai[a] == a) return a;
        return pai[a] = find(pai[a]);
    }

    void uu(int a, int b){
        a = find(a), b = find(b);
        if(a == b) return;
        if(rank[a] > rank[b]) swap(a,b);
        rank[b] += rank[a];
        pai[a] = b;
    }

};
```

2.5 Mo

```
struct Mo{
  struct Query {
    int l, r, idx, block_size;
    bool operator<(Query other) const{
      return make_pair(l / block_size, r) <
        make_pair(other.l / block_size, other.r);
    }
  };

  //TODO: declare the data structures
  Mo(){

  }

  void add(int idx){
    //TODO: add an element to the data structure
  }

  void remove(int idx){
    //TODO: remove an element from the data structure
  }

  int get_answer(){
    //TODO: get answer from the data structure
  }

  vector<int> solve(vector<Query> queries) {
    vector <int> answers(queries.size());
    sort(queries.begin(), queries.end());

    //TODO: initialize data structue

    int cur_l = 0;
    int cur_r = -1;
    for (Query q : queries) {
      while (cur_l > q.l){
        cur_l--;
        add(cur_l);
      }
      while (cur_r < q.r){
        cur_r++;
        add(cur_r);
      }
      while (cur_l < q.l){
        remove(cur_l);
        cur_l++;
      }
      while (cur_r > q.r){
        remove(cur_r);
        cur_r--;
      }
      answers[q.idx] = get_answer();
    }
    return answers;
  }
};
```

Capítulo 3

Grafos

3.1 Pontos de Articulação e Pontes

```
template<typename T> struct ArticPont
{
    int n;
    int count = 0;
    vector<int> marc, tin, low, artic;
    vector<vector<int>> grafo;
    vector<pair<int,int>> bridges;

    ArticPont(int n) : n(n), grafo(n+1), marc(n+1), tin(n+1), low(n+1),
    artic(n+1) {}

    void add_edge(int a, int b){
        grafo[a].push_back(b);
        grafo[b].push_back(a);
    }

    void dfs(ll x, ll p){
        marc[x] = 1;
        tin[x] = low[x] = ++count;
        ll children = 0;
        for(ll viz : grafo[x]){
            if(viz == p) continue;
            if(marc[viz]){
                low[x] = min(low[x], tin[viz]);
            }else{
                dfs(viz,x);
                low[x] = min(low[x], low[viz]);
                if(low[viz] > tin[x]){
                    bridges.push_back({min(viz,x), max(viz, x)});
                }
                if(low[viz] >= tin[x] && p) artic[x] = 1;
                children++;
            }
        }
        if(!p && children>1) artic[x] = 1;
    }

    void find_brig_and_artc(){
        for(ll i=1; i<=n; i++){
            if(!marc[i]) dfs(i,0);
        }
    }
};
```

3.2 Ordenação Topológica

```
struct TopoSort
{
    int n;
    vector<int> grau;
    vector<vector<int>> grafo;

    TopoSort(int n): n(n), grau(n+1), grafo(n+1){}

    void add_edge(int a, int b){
        grau[b]++;
```

```
        grafo[a].push_back(b);
    }

    vector<int> top_sort(){
        vector<int> resp;
        queue<int> fila;
        for(int i=1; i<=n;i++){
            if(!grau[i])fila.push(i);
        }
        while (!fila.empty())
        {
            int u = fila.front();
            resp.push_back(u);
            fila.pop();
            for(int viz : grafo[u]){
                grau[viz]--;
                if(!grau[viz])fila.push(viz);
            }
        }
        if(resp.size() < n){
            return {};
        }else{
            return resp;
        }
    }
};
```

3.3 Componentes Fortemente Conexos: Algoritmo de Kosaraju

```
template<typename T> struct Kosa{

    int n, cont;
    vector<int> marc, ord, comp;
    vector<vector<int>> grafo, rgrafo,scc;

    Kosa(int n) : n(n), marc(n+1), grafo(n+1), rgrafo(n+1),
    comp(n+1), scc(n+1) {}

    void add_edge(int a, int b){
        grafo[a].push_back(b);
        rgrafo[b].push_back(a);
    }

    void dfs1(int v){
        marc[v] = 1;
        for(auto viz : grafo[v]){
            if(!marc[viz]) dfs1(viz);
        }
        ord.push_back(v);
    }

    void dfs2(int v, int c){
        comp[v] = c;
        for(auto viz : rgrafo[v]){
            if(!comp[viz]) dfs2(viz, c);
        }
    }

    void build(){

        cont = 0;

        for(int i = 1; i <=n ;i++){
            if(!marc[i]) dfs1(i);
        }

        reverse(ord.begin(), ord.end());
```

```
        for(int v : ord){
            if(!comp[v]){
                dfs2(v, ++cont);
            }
        }

        for(int i = 1; i <=n; i++){
            for(int j : grafo[i]){
                if(comp[i] == comp[j]) continue;
                scc[comp[i]].push_back(comp[j]);
            }
        }
    }
};
```

3.4 2-SAT

```
template<typename T> struct SAT2{

    int n, cont;
    vector<char> resp;
    vector<int> marc, ord, comp;
    vector<vector<int>> grafo, rgrafo,scc;

    SAT2(int n) :  n(n), marc(2*n+2), grafo(2*n+2), rgrafo(2*n+2),
    comp(2*n+2), resp(2*n + 2){}

    void add_edge(int sx, int x, int sy, int y){ // '+' = 1, '-' = 0

        grafo[y+n*(!sy)].push_back(x+n*sx); //~y -> x
        grafo[x+n*(!sx)].push_back(y+n*sy); //~x -> y

        rgrafo[x+n*sx].push_back(y+n*(!sy));
        rgrafo[y+n*sy].push_back(x+n*(!sx));

    }

    void dfs1(int v){
        marc[v] = 1;
        for(auto viz : grafo[v]){
            if(!marc[viz]) dfs1(viz);
        }
        ord.push_back(v);
    }

    void dfs2(int v, int c){
        comp[v] = c;
        for(auto viz : rgrafo[v]){
            if(!comp[viz]) dfs2(viz, c);
        }
    }

    void build(){

        cont = 0;

        for(int i = 1; i <=2*n ;i++){
            if(!marc[i]) dfs1(i);
        }

        reverse(ord.begin(), ord.end());

        for(int v : ord){
            if(!comp[v]){
                dfs2(v, ++cont);
            }
        }

        bool can = true;
        for(int i= 1; i <=n; i++){
            if(comp[i] == comp[i+n]) can = false;
            resp[i]=comp[i]<comp[i+n]?'+':'-';
            //positiva é comp[i+n], está escolhendo a variavel que não
            //tem um caminho de implicação que resulta em impossivel
        }

        if(can){
            for(int i = 1; i <=n; i++){
                cout << resp[i] << " ";
            }
        }else{
            cout << "IMPOSSIBLE" << endl;
        }
    }
```

```
};
```

3.5 Caminho mínimo: Algoritmo de Dijkstra

```
template<typename T> struct Dijkstra
{
    ll INF = 1e18;

    int n;
    vector<ll> dist;
    vector<vector<pair<int,int>>> g;

    Dijkstra(int n) : n(n), dist(n+1,INF), g(n+1) {}

    void addEdge(ll v, ll u, ll p){
        g[v].push_back({u,p});
        g[u].push_back({v,p});
    }

    void run(ll v){

        priority_queue<pair<ll,ll>, vector<pair<ll,ll>>,
        greater<pair<ll,ll>>> fila;

        fila.push({0,v});

        while (!fila.empty())
        {
            ll vert = fila.top().second;
            ll price = fila.top().first;
            fila.pop();

            if(dist[vert] != INF) continue;

            dist[vert] = price;

            for(auto viz : g[vert]){
                ll nxt = viz.first;
                ll cost = viz.second;
                fila.push({price + cost, nxt});
            }
        }
    }
};
```

3.6 Caminho mínimo: Algoritmo de Bellman Ford

```
struct Edge{
    int v, u, cost;
    Edge(int v, int u, int cost): v(v), u(u), cost(cost) {}
};

template<typename T> struct Ford
{
    const ll INFL = 1e18;
    int n, m;
    vector<Edge> edges;
    vector<ll> dist;

    Ford(int n, int m) : n(n), m(m), dist(n+1, INFL) {}

    void add_edge(int v, int u, int cost){
        edges.emplace_back(v,u,cost);
    }

    ll bellman(int s, int t){
        dist[s] = 0;

        for(int k=1; k < n; k++){
            for(Edge e : edges){
                int a = e.v, b = e.u, c = e.cost;
                if(dist[a] != MINFL && dist[b] > dist[a] + c){
                    dist[b] = dist[a] + c;
                }
            }
        }

        //Detecção de ciclo negativo
        for(Edge e : edges){
            int a = e.v, b =e.u, c=e.cost;
            if(dist[a] != MINFL && dist[b] > dist[a]+c){
                return -1;
            }
```

```
        }

        return dist[t];

    }
};
```

3.7 Caminho mínimo: Algoritmo de Floyd-Warshall

```
template<typename T> struct FloydWarshall
{
    const int MAXN = 500;
    cont ll INF = 1e18;
    vector dist(maxn, vector<ll>(maxn, INF));

    void floydWarshall( ){

        for(int i = 0; i < MAXN; i++)dist[i][i] = 0;

        for(int k = 1; k < MAXN; k++)
            for(int i = 1; i < MAXN; i++)
                for(int j = 1; j < MAXN; j++){
                    if(dist[i][k] < INF && dist[k][j] < INF)
                        dist[i][j] = min(dist[i][j], dist[i][k] +
                                                    dist[k][j]);
                }
    }
};
```

3.8 Árvore Geradora Mínima: Algoritmo de Kruskal

```
template<typename T> struct Kruskal
{
    void MinTree(){
        sort(arestas.begin(), arestas.end());
        for(auto arr : arestas){
            int p = arr[0], a = arr[1], b = arr[2];
            if(find(a) != find(b)){
                uu(a,b);
                //adicionar aresta
            }
        }
    }
};
```

3.9 Caminho/Circuito Euleriano

```
 * Returns a list of nodes in the Eulerian path/cycle with src at both start and
 * empty list if no cycle/path exists.
 * To get edge indices back, add .second to s and ret.
 * Time: O(V + E)
 * Status: stress-tested
 *
 * Condições para a existencia de um caminho/cicutio euleriano:
 *
 *          |        Direcionado             |         Não Direcionado
 * ---------+--------------------------------+--------------------------
 *          | "existem 0 ou 1 vértices       |
 * Caminho  |  com difrença 1 entre grau     | "existem 0 ou 2 vértices de grau i
 *          |  de entrada e saida"           |
 * ---------+--------------------------------+-----------------------------------
 *          | "Grau de entrada e saida       |
 * Circuito |  de todos os vértices          | "não existe vértice de grau impar"
 *          |  são iguais"                   |
 *

 */
vector<int> eulerWalk(vector<vector<pii>>& gr, int nedges, int src=0) {
        int n = gr.size();
        vector<int> D(n), its(n), eu(nedges), ret, s = {src};
        D[src]++; // to allow Euler paths, not just cycles
        while (!s.empty()) { ///start-hash
                int x = s.back(), y, e, &it = its[x], end = int(gr[x].size());
                if (it == end){ ret.push_back(x); s.pop_back(); continue; }
                tie(y, e) = gr[x][it++];
                if (!eu[e])
                        D[x]--, D[y]++, eu[e] = 1, s.push_back(y);
        } ///end-hash
        for(auto &x : D) if (x < 0 || int(ret.size()) != nedges+1) return {};
        return {ret.rbegin(), ret.rend()};
}
```

3.10 Bipartite Matching: Kuhn

```
// O(N*M)
template<typename T> struct bm_t
{
    int N, M, T;
    vector<vector<int>> grafo;
    vector<int> match, seen;
    bm_t(int a, int b) : N(a), M(a+b), T(0), grafo(M),
                         match(M, -1), seen(M, -1) {}

    void add_edge(int a, int b){
        grafo[a].push_back(b + N);
    }

    bool dfs(int cur){
        if(seen[cur] == T) return false;
        seen[cur] = T;
        for(int nxt : grafo[cur]) if(match[nxt] == -1){
            match[nxt] = cur;
            match[cur] = nxt;
            return true;
        }
        for(int nxt : grafo[cur]) if(dfs(match[nxt])){
            match[nxt] = cur;
            match[cur] = nxt;
            return true;
        }
        return false;
    }

    int solve(){
        int res = 0;
        for(int cur = 1; cur;){
            cur = 0; ++T;
            for(int i = 0; i < N; ++i) if(match[i] == -1)
                cur += dfs(i);
            res += cur;
        }
        return res;
    }
};
```

3.11 Bipartite Matching: Hopcroft Karp

```
// O(\sqrt{V}E)
using vi = vector<int>;
bool dfs(int a, int L, const vector<vi> &g, vi &btoa, vi &A, vi &B) {
        if (A[a] != L) return 0;
        A[a] = -1;
        for(auto &b : g[a]) if (B[b] == L + 1) {
                B[b] = 0;
                if (btoa[b] == -1 || dfs(btoa[b], L+1, g, btoa, A, B))
                        return btoa[b] = a, 1;
        }
        return 0;
}
int hopcroftKarp(const vector<vi> &g, vi &btoa) {
        int res = 0;
        vector<int> A(g.size()), B(int(btoa.size())), cur, next;
        for (;;) {
                fill(A.begin(), A.end(), 0), fill(B.begin(), B.end(), 0);
                cur.clear();
                for(auto &a : btoa) if (a != -1) A[a] = -1;
                for (int a = 0; a < g.size(); ++a)
                if (A[a] == 0) cur.push_back(a);
                for (int lay = 1;; ++lay) {
                        bool islast = 0; next.clear();
                        for(auto &a : cur) for(auto &b : g[a]) {
                                if (btoa[b] == -1) B[b] = lay, islast = 1;
                                else if (btoa[b] != a && !B[b])
                                        B[b] = lay, next.push_back(btoa[b]);
                        }
                        if (islast) break;
                        if (next.empty()) return res;
                        for(auto &a : next) A[a] = lay;
                        cur.swap(next);
                }
                for(int a = 0; a < int(g.size()); ++a)
                        res += dfs(a, 0, g, btoa, A, B);
        }
}
```

3.12 Fluxo Máximo: Dinic

```
// O(min(V*max_flow, V²*E))
// Grafo com capacidades 1: O(min(E*sqrt(E), E*V^(2/3)))
// Todo vértice tem grau de entrada ou saída 1: O(E*sqrt(V))
template<typename T> struct Edge{
    int v, u;
    ll cap, flow = 0;
    Edge(int v, int u, ll cap) : v(v), u(u), cap(cap) {}
};

template<typename T> struct Dinic{
    ll INFL = 1e18;
    int n, m = 0, s, t;
    vector<Edge> edges;
    vector<vector<int>> vec;
    vector<int> lv, pos;
    queue<int> fila;

    Dinic(int n, int s, int t): n(n), s(s), t(t), vec(n+1), lv(n+1),
    pos(n+1) {}

    void add_edge(int v, int u, ll cap){
        edges.emplace_back(v, u, cap);
        edges.emplace_back(u,v,0);
        vec[v].push_back(m);
        vec[u].push_back(m+1);
        m+=2;
    }

    int bfs(){
        while (!fila.empty())
        {
            int v = fila.front();
            fila.pop();
            for(int i : vec[v]){
                if(edges[i].cap - edges[i].flow < 1) continue;
                if(lv[edges[i].u] != -1) continue;

                lv[edges[i].u] = lv[v] + 1;
                fila.push(edges[i].u);
            }
        }
        return lv[t] !=-1;
    }

    ll dfs(int v, ll menor){
        if(!menor) return 0;
        if(v == t) return menor;

        for(int& j=pos[v]; j < (int)vec[v].size(); j++){
            int i=vec[v][j];
            int u=edges[i].u;

            if(lv[v]+1 != lv[u] || edges[i].cap - edges[i].flow < 1) continue;
            ll agr=dfs(u, min(menor, edges[i].cap - edges[i].flow));
            if(!agr) continue;

            edges[i].flow += agr;
            edges[i^1].flow -= agr;

            return agr;
        }
        return 0;
    }

    ll max_flow(){
        ll flow = 0;
        while (1)
        {
            fill(lv.begin(), lv.end(), -1);
            lv[s] = 0;
            fila.push(s);
            if(!bfs()) break;
            fill(pos.begin(), pos.end(), 0);
            while(ll atual=dfs(s,INFL)) flow += atual;
        }
        return flow;
    }

    void recap(){
        for(int i=0; i<(int)edges.size(); i+=2){
            if(lv[edges[i].v]>=0 && lv[edges[i].u] == -1){
                cout << edges[i].v << " " << edges[i].u << endl;
```

```
            }
        }
    }
};
```

3.13 Min Cost - Max Flow

```
//Min-cost max-flow. Assumes there is no negative cycle.
//Time: O (F (V + E)logV ), being F the amount of flow.

template<class flow_t, class cost_t> struct min_cost {
    static constexpr flow_t FLOW_EPS = flow_t(1e-10);
    static constexpr flow_t FLOW_INF = numeric_limits<flow_t>::
        max();
    static constexpr cost_t COST_EPS = cost_t(1e-10);
    static constexpr cost_t COST_INF = numeric_limits<cost_t>::
        max();
    int n, m{}; vector<int> ptr, nxt, zu;
    vector<flow_t> capa; vector<cost_t> cost;

    min_cost(int N) : n(N),ptr(n,-1),dist(n),vis(n),pari(n) {}

    void add_edge(int u, int v, flow_t w, cost_t c) {
        nxt.push_back(ptr[u]); zu.push_back(v); capa.push_back(w);
        cost.push_back(c); ptr[u] = m++;
        nxt.push_back(ptr[v]); zu.push_back(u); capa.push_back(0);
        cost.push_back(-c); ptr[v] = m++;
    }

    vector<cost_t> pot, dist; vector<bool> vis; vector<int> pari;
    vector<flow_t> flows; vector<cost_t> slopes;
    // You can pass t = =1 to find a shortest
    void shortest(int s, int t) {//path to each vertex . // hash=1
        using E = pair<cost_t, int>;
        priority_queue<E, vector<E>, greater<E>> que;
        for(int u = 0; u < n; ++u){dist[u]=COST_INF; vis[u]=false;}
        for (que.emplace(dist[s] = 0, s); !que.empty(); ) {
            const cost_t c = que.top().first;
            const int u = que.top().second; que.pop();
            if (vis[u]) continue;
            vis[u] = true; if (u == t) return;
            for (int i = ptr[u]; ~i; i = nxt[i]) if (capa[i] > FLOW_EPS) {
                const int v = zu[i];
                const cost_t cc = c + cost[i] + pot[u] - pot[v];
                if(dist[v] > cc){que.emplace(dist[v]=cc,v);pari[v]=i;}
            }
        }
    }
     // hash=1 = 89f16a
    auto run(int s, int t, flow_t limFlow = FLOW_INF) { // hash=2
        pot.assign(n, 0); flows = {0}; slopes.clear();
        while (true) {
            bool upd = false;
            for (int i = 0; i < m; ++i) if (capa[i] > FLOW_EPS) {
                const int u = zu[i ^ 1], v = zu[i];
                const cost_t cc = pot[u] + cost[i];
                if(pot[v] > cc + COST_EPS) { pot[v] = cc; upd = true; }
            } if (!upd) break;
        }
        flow_t flow = 0; cost_t tot_cost = 0;
        while (flow < limFlow) {
            shortest(s, t); flow_t f = limFlow - flow;
            if (!vis[t]) break;
            for(int u = 0; u < n; ++u)pot[u] += min(dist[u],dist[t]);
            for (int v = t; v != s; ) { const int i = pari[v];
                if (f > capa[i]) { f = capa[i]; } v = zu[i^1];
                }
            for (int v = t; v != s; ) { const int i = pari[v];
                capa[i] -= f; capa[i^1] += f; v = zu[i^1];
                }
            flow += f; tot_cost += f * (pot[t] - pot[s]);
            flows.push_back(flow); slopes.push_back(pot[t] - pot[s]);
        } return make_pair(flow, tot_cost);
    } // hash=2 = 285527
};
```

3.14 Weighted Matching: Hungarian

Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j].
returns (min cost, match), where L[i] is matched with R[match[i]].
Negate costs for max cost.
Time: O(N^2*M)

```cpp
template<class cost_t> pair<cost_t, vector<int>> hungarian
(const vector<vector<cost_t>> &a)
{
    int n = a.size() + 1, m = a[0].size() + 1;

    vector<int> p(m), ans(n - 1);
    vector<cost_t> u(n), v(m);
    for(int i = 1; i < n; ++i) {
        p[0] = i; int j0 = 0;
        vector<cost_t> dist(m, 1e9);
        vector<int> pre(m, -1);
        vector<bool> done(m + 1);
        do {
            done[j0] = true;
            int i0 = p[j0], j1;
            cost_t delta = 1e9;
            for(int j = 1; j < m; ++j) if (!done[j]) {
                auto cur = a[i0-1][j-1] - u[i0] - v[j];
                if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
                if (dist[j] < delta) delta = dist[j], j1 = j;
            }
            for(int j = 0; j < m; ++j)
                if (done[j]) u[p[j]] += delta, v[j] -= delta;
                else dist[j] -= delta;
            j0 = j1;
        } while (p[j0]);
        while (j0) {
            int j1 = pre[j0]; p[j0] = p[j1], j0 = j1;
        }
    }
    for(int j = 1; j < m; ++j) if (p[j]) ans[p[j]-1] = j-1;
    return {-v[0], ans};
}
```

3.15 Binary Lifting + LCA

Time: O($N \log N$) for Binary Lifting
O($\log N$) for LCA

```cpp
struct Arvore{

    // jumps[k][i] e o resultado de 2^k passos a partir de i
    // prof[i] = profundidade do vertice i
    // pai[i] = pai do vertice i

    int n , raiz ;
    vector<vector<int>> jumps ;
    vector<int> pai , prof ;


    Arvore(int num, int raiz){
        n = num ;
        raiz = raiz ;
        jumps.resize(33,vector<int>(n+1,-1)) ;
        pai.resize(n+1) ;
        prof.resize(n+1) ;
    }

    void binery_lifting(vector<int> &pai, int raiz){ // O(n)

        for(int i=1 ; i<=n ; i++){
            if(i==raiz) continue ;
            jumps[0][i] = pai[i] ;
        }

        for( int k=1 ; k<33 ; k++ ){
            for( int i=2 ; i<=n ; i++ ){
                if(jumps[k-1][i]==-1) continue ;
                jumps[k][i] = jumps[k-1][jumps[k-1][i]] ;
            }
        }

    }

    pair<int,int> nivelar(int u , int v){ //O(logn)

        if( prof[u] > prof[v] ){

            for( int i = 32 ; i >= 0 ; i-- ){
                if( (1LL<<i) & abs(prof[u]-prof[v]) ) u = jumps[i][u] ;
            }

        }
```

```
        else if( prof[u] < prof[v] ){

            for( int i = 32 ; i >= 0 ; i-- ){
                if( (1LL<<i) & abs(prof[u]-prof[v]) ) v = jumps[i][v] ;
            }

        }

        return {u,v} ;

    }

    int lca(int u, int v){ // O(logn)

        pair<int,int> temp = nivelar(u,v) ;
        int a = temp.first ; // u nivelado
        int b = temp.second ; // v nivelado

        if( a == b ) return a ;

        for( int i = 32 ; i >= 0 ; i-- ){
            if( jumps[i][a] != jumps[i][b] ){
                a = jumps[i][a] ;
                b = jumps[i][b] ;
            }
        }

        if(pai[a]==pai[b] && pai[a] != -1) return pai[a] ;

        return -1 ;

    }

} ;
```

Capítulo 4

Matemática

4.1 Crivo de Eratóstenes

```
struct Sieve{
  int maxn;
  vector <int> is_prime, min_div;

  Sieve(int n){
    this->maxn = n;
    is_prime.assign(n+1, 1);
    min_div.resize(n+1);

    for(int i = 0; i <= n; i++)
      min_div[i] = i;

    is_prime[0] = is_prime[1] = 0;
    for (int i = 2; i <= n; i++) {
      if (is_prime[i] && (long long)i * i <= n) {
        for (int j = i * i; j <= n; j += i){
          if(is_prime[j]) min_div[j] = i;
          is_prime[j] = false;
        }
      }
    }
  }

  vector <pair<int,int>> factorize(int n){
    assert(n <= maxn);
    vector <pair<int,int>> fact;
    while(n > 1){
      if(fact.empty() || fact.back().first != min_div[n]){
        fact.push_back({min_div[n], 1});
      }else{
        fact.back().second += 1;
      }
      n /= min_div[n];
    }
    return fact;
  }
};
```

4.2 Exponenciação de Matriz

```
const int D = 2;
const int MOD = 1000000007;
struct Matriz{
  int mat[D][D];
  int* operator[](int i){
    return mat[i];
  }
  Matriz operator*(Matriz oth){
    Matriz res;
    for(int i=0; i<D; i++){
      for(int j=0; j<D; j++){
        res[i][j] = 0;
        for(int k=0; k<D; k++)
          res[i][j] = (res[i][j]+(mat[i][k]*1LL*oth[k][j])%MOD)%MOD;
      }
    }
```

```
    return res;
  }
  Matriz exp(long long e){
    Matriz res;
    for(int i=0; i<D; i++)
      for(int j=0; j<D; j++)
        res[i][j] = (i==j);
    Matriz base = *this;
    while(e > 0){
      if(e & 1LL)
        res = res * base;
      base = base*base;
      e = e>>1;
    }
    return res;
  }
};
```

4.3 Exponenciação Rápida

Para calcular o inverso multiplicativo, bastar elevar o número a MOD-2

```
int exp(int b, int e, int m=MOD){ // O(logE)
    int res=1;
    while(e){
        if(e&1) res=(res*b)%m;
        b=(b*b)%m;
        e>>=1;
    }
    return res;
}
```

4.4 Fatoração em Primos: Lenta

```
vector<int> fact(int x){ // O(sqrt(N))
    vector<int> factors;

    if(x%2==0) factors.pb(2);
    while(x%2==0) x>>=1;

    for(int i=3; i*i<=x; i+=2){
        if(x%i==0) factors.pb(i);
        while(x%i==0) x/=i;
    }

    if(x>2) factors.pb(x);

    return factors;
}
```

4.5 Fatoração em Primos: Pollard Rho

```
ull modmul(ull a, ull b, ull M){
    ll ret = a * b - M * ull(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod){
    ull ans = 1;
    for(; e; b = modmul(b, b, mod), e /= 2){
        if(e & 1) ans = modmul(ans, b, mod);
    }
    return ans;
}
ull pollard(ull n){ // O(N^(1/4))
    auto f = [n](ull x, ull k) { return modmul(x, x, n) + k; };
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    while(t++ % 40 || __gcd(prd, n) == 1){
        if(x == y) x = ++i, y = f(x, i);
        if((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
        x = f(x, i), y = f(f(y, i), i);
    }
    return __gcd(prd, n);
}
bool isPrime(ull n){ // O(logN)
    if(n<2 || n % 6 % 4 != 1) return (n | 1) == 3;
    vector<ull> A = {2, 325, 9375, 28178, 450775, 9780504,
    1795265022};
    ull s = __builtin_ctzll(n-1), d = n >> s;
    for(ull a : A){
        ull p = modpow(a%n, d, n), i = s;
        while(p!=1 && p!=n-1 && a%n && i--){
            p = modmul(p, p, n);
        }
```

```
        if(p!=n-1 && i!=s) return 0;
    }
    return 1;
}
vector<ull> fact(ull n){
    if(n==1) return {};
    if(isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = fact(x), r = fact(n/x);
    l.insert(l.end(), r.begin(), r.end());
    return l;
}
```

4.6 FFT

```
template<typename dbl> struct cplx{
    dbl x, y; using P = cplx;
    cplx(dbl x_=0, dbl y_=0) : x(x_), y(y_) {}
    friend P operator+(P a, P b) { return P(a.x+b.x, a.y+b.y); }
    friend P operator-(P a, P b) { return P(a.x-b.x, a.y-b.y); }
    friend P operator*(P a, P b) { return P(a.x*b.x - a.y*b.y,
    a.x*b.y + a.y*b.x); }
    friend P conj(P a) { return P(a.x, -a.y); }
    friend P inv(P a) { dbl n = (a.x*a.x+a.y*a.y); return P(a.x/n,-a.y/n); }
};
template<typename T> struct root_of_unity {};
template<typename dbl> struct root_of_unity<cplx<dbl>>{
    static cplx<dbl> f(int k){
        static const dbl PI = acos(-1); dbl a = 2*PI/k;
        return cplx<dbl>(cos(a), sin(a));
    }
};
typedef cplx<double> cd;
inline int nxt_pow2(int s) { return 1 << (s>1 ? 32-__builtin_clz(s-1) : 0); }
template<typename T> struct FFT{
    vector<T> rt; vector<int> rev;
    FFT() : rt(2, T(1)) {}
    void init(int N){
        N = nxt_pow2(N);
        if(N>int(rt.size())){
            rev.resize(N); rt.reserve(N);
            for(int a=0; a<N; a++) rev[a] = (rev[a/2] | ((a&1)*N)) >> 1;
            for(int k=int(rt.size()); k<N; k*=2){
                rt.resize(2*k);
                T z = root_of_unity<T>::f(2*k);
                for(int a=k/2; a<k; a++) rt[2*a] = rt[a], rt[2*a+1] = rt[a]*z;
            }
        }
    }
    void fft(vector<T>& xs, bool inverse) const{
        int N = int(xs.size());
        int s = __builtin_ctz(int(rev.size())/N);
        if(inverse) reverse(xs.begin()+1, xs.end());
        for(int a=0; a<N; a++){
            if(a<(rev[a] >> s)) swap(xs[a], xs[rev[a] >> s]);
        }
        for(int k=1; k<N; k*=2){
            for(int a=0; a<N; a+=2*k){
                int u = a, v = u+k;
                for(int b=0; b<k; b++, u++, v++){
                    T z = rt[b+k]*xs[v];
                    xs[v] = xs[u]-z, xs[u] = xs[u]+z;
                }
            }
        }
        if(inverse) for(int a=0; a<N; a++) xs[a] = xs[a]*inv(T(N));
    }
    vector<T> convolve(vector<T> as, vector<T> bs){
        int N = int(as.size()), M = int(bs.size());
        int K = N+M-1, S = nxt_pow2(K); init(S);
        if(min(N, M)<=64){
            vector<T> res(K);
            for(int u=0; u<N; u++){
                for(int v=0; v<M; v++){
                    res[u+v] = res[u+v]+as[u]*bs[v];
                }
            }
            return res;
        }else{
            as.resize(S), bs.resize(S);
            fft(as, false); fft(bs, false);
            for(int i=0; i<S; i++) as[i] = as[i]*bs[i];
            fft(as, true); as.resize(K); return as;
        }
```

```
    }
};
/*
vector<cd> a(n), b(m);
FFT<cd> fft;
vector<cd> mult = fft.convolve(a, b);
(mult[i].x + 0.5) or -0.5
(roundll(mult[i].x))
*/
```

4.7 Basis

```
struct Basis{
  vector <int> basis;
  Basis(){

  }
  Basis(int x){
    add(x);
  }
  Basis operator+(Basis other) const{
    Basis res;
    for(int x : basis){
      res.add(x);
    }
    for(int x : other.basis){
      res.add(x);
    }
    return res;
  }
  void add(int x){
    for(auto& i : basis){
      x = min(x, x^i);
    }
    if(x){
      basis.push_back(x);
    }
  }
};
```

Capítulo 5

Geometria Computacional

Capítulo 6

Strings

6.1 Prefix Function

```
struct PrefixFunction{
  vector<int> get_pi(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
      int j = pi[i-1];
      while (j > 0 && s[i] != s[j])
        j = pi[j-1];
      if (s[i] == s[j])
        j++;
      pi[i] = j;
    }
    return pi;
  }
};
```

6.2 Hashing

```
struct HashedString {
   long long B, M;
   vector<long long> pow = {1}; // pow[i] contains B^i % M
   vector<long long> p_hash; // p_hash[i] is the hash of the first i
   //characters of the given string

  /*
  for random base:
  mt19937 rng((uint32_t)chrono::steady_clock::now().time_since_epoch().count());
  const long long B = uniform_int_distribution<long long>(0, M - 1)(rng);
  */

    HashedString(const string &s, long long mod = 1e9 + 9,
    long long base = 9973){
        B = base, M = mod;
        while (pow.size() < s.size())
                pow.push_back((pow.back() * B) % M);
        p_hash.resize(s.size()+1);
        p_hash[0] = 0;
        for (int i = 0; i < s.size(); i++) {
                p_hash[i + 1] = ((p_hash[i] * B) % M + s[i]) % M;
        }
    }

    long long get_hash(int start, int end) {
        long long raw_val = (p_hash[end + 1] - (p_hash[start] *
        pow[end - start + 1]));
        return (raw_val % M + M) % M;
    }
};
```

Capítulo 7

Variados

7.1 Ternary Search

< for maximum and > for minimum value

```
int ternary(int l, int r){
    while(r-l>=5){
        int mid=(l+r)>>1;
        if(f(mid)<f(mid+1)){
            l=mid;
        }else{
            r=mid+1;
        }
    }
    for(int i=l+1; i<=r; i++){
        if(f(l)<f(i)) l=i;
    }
    return l;
}
```

```
double ternary(double l, double r){
    int cont=200;
    while(cont--){
        double m1=l+(r-l)/3;
        double m2=r-(r-l)/3;
        double f1=f(m1);
        double f2=f(m2);
        if(f1<f2){
            l=m1;
        }else{
            r=m2;
        }
    }
    return l;
}
```

7.2 Digit DP

```
ll solve(string &s, int i, int tight, int last, int started){
//numbers that no two adjacent digits are the same
    if(i==(int)s.size()) return 1;

    if(!tight && dp[i][last][started]!=-1) return dp[i][last][started];

    int lim=(tight?s[i]-'0':9);

    ll resp=0;
    for(int j=0; j<=lim; j++){
        if(started && j==last) continue;
        resp+=solve(s, i+1, tight&(j==lim), j, (started|j)>0);
    }

    if(!tight) return dp[i][last][started]=resp;
    return resp;
}

ll func(ll a, ll b){
    string agr1=to_string(a-1);
    memset(dp, -1, sizeof(dp));
    ll ans1 = solve(agr1, 0, 1, 10, 0);

    string agr2=to_string(b);
    memset(dp, -1, sizeof(dp));
    ll ans2 = solve(agr2, 0, 1, 10, 0);

    return ans2-ans1;
}
```

7.3 Busca Binária Paralela

```
int l[mxn], r[mxn], resp[mxn];
vector<int> meio[mxq];

void build(){
    //remember to check l and r limits (maybe l can be 0)
    for(int i=0; i<n; i++){
        l[i]=1, r[i]=q, resp[i]=-1;
    }
}

void bb(){
    int ok=1;
    while(ok){
        ok=0;
        //restart the structure
        for(int i=0; i<n; i++){
            if(l[i]<=r[i]) meio[(l[i]+r[i])>>1].pb(i);
        }
        for(int i=1; i<=q; i++){
            //do the current update
            while(!meio[i].empty()){
                ok=1;
                int k = meio[i].back();
                meio[i].pop_back();
                if(/*check condition*/){
                    resp[k]=i;
                    r[k]=i-1;
                }else{
                    l[k]=i+1;
                }
            }
        }
    }
}
```

7.4 Mochila Raiz

```
vector<int> knapsack(const vector<int> &count) {
    // we have `count[coin]` coins of cost `coin`;
    // now we create some super-coins, so that all possible sums are still reach
    //with the same number of coins
    // super-coin is a union of `count` coins with the same cost
    // we record in `counts[sum]` all possible numbers of coins to create
    //one super-coin of cost `sum`
    vector<vector<int>> counts(count.size());
    for (int i = 1; i < count.size(); ++i) {
        int cur = count[i], j = i;
        for (; cur >= 3; j *= 2) {
            int carry = (cur - 1) / 2;
```

```
            for (int x = 0; x < cur - 2 * carry; ++x) {
                counts[j].push_back(j / i);
            }
            cur = carry;
        }
        for (int x = 0; x < cur; ++x) {
            counts[j].push_back(j / i);
        }
    }
    vector<int> knapsack(count.size(), 1e9); // `knapsack[sum]` is minimum number of coins to achieve `sum`
    knapsack[0] = 0;
    for (int i = 1; i < counts.size(); ++i) {
        for (int number: counts[i]) {
            for (int j = (int) knapsack.size() - 1 - i; j >= 0; --j) {
                knapsack[j + i] = min(knapsack[j + i], knapsack[j] + number);
            }
        }
    }
    return knapsack;
}
```